Investigación: Pipeline básico de CI/CD utilizando GitHub Actions

Objetivo

Investigar y experimentar con un pipeline básico de Integración Continua y Entrega Continua (CI/CD) utilizando GitHub Actions, aplicándolo a un proyecto desarrollado con el framework Flutter.

1. ¿Qué es CI/CD?

CI/CD es un conjunto de prácticas utilizadas en el desarrollo de software para automatizar diferentes etapas del ciclo de desarrollo, principalmente la integración, comprobación, construcción y entrega de aplicaciones.

CI significa Continuous Integration o Integración Continua. CD significa Continuous Delivery o Entrega Continua. El objetivo es permitir que los cambios puedan integrarse y comprobarse de forma frecuente, reduciendo errores y facilitando la entrega de nuevas versiones.

2. ¿Qué significa Integración Continua?

La Integración Continua (CI) es una práctica de desarrollo en la que los cambios realizados por los desarrolladores se integran frecuentemente en un repositorio compartido. Cada cambio puede activar procesos automáticos para comprobar que el código continúa funcionando correctamente.

Beneficios: detectar errores rápidamente, automatizar pruebas, evitar comprobaciones manuales repetitivas, mejorar la calidad del código y facilitar el trabajo colaborativo.

3. ¿Qué significa Entrega Continua?

La Entrega Continua (CD) consiste en mantener el software en condiciones de poder ser entregado o desplegado de manera rápida y confiable. Después de validar el código mediante pruebas y otras comprobaciones, puede prepararse para su entrega.

Entrega Continua no necesariamente significa despliegue automático a producción; el proyecto puede quedar preparado para publicación y requerir una aprobación posterior.

4. ¿Qué es GitHub Actions?

GitHub Actions es una plataforma de automatización integrada en GitHub que permite crear procesos automáticos para proyectos de software. Puede utilizarse para compilar aplicaciones, ejecutar pruebas, analizar código, instalar dependencias, generar archivos y automatizar procesos de CI/CD.

Los procesos se definen mediante archivos YAML, normalmente dentro de .github/workflows/.

5. ¿Qué es un Workflow?

Un workflow es un proceso automatizado configurable que contiene uno o varios trabajos (jobs). Se define mediante un archivo YAML y puede ejecutarse cuando ocurre un evento en el repositorio, como un push o un

pull request.

6. ¿Qué es un Job?

Un job es un conjunto de pasos que se ejecutan como una unidad dentro de un workflow. Un workflow puede tener uno o varios jobs. Cada job se ejecuta en un runner y puede realizar tareas como analizar código, compilar o ejecutar pruebas.

7. ¿Qué es un Step?

Un step es una tarea individual dentro de un job. Puede ejecutar un comando mediante run o utilizar una acción existente mediante uses.

8. Estructura básica de un archivo de configuración de GitHub Actions

Los workflows utilizan archivos .yml o .yaml y normalmente se almacenan en .github/workflows/. Sus elementos principales son:

name: nombre del workflow.
on: eventos que activan el workflow.
jobs: trabajos que realizará.
runs-on: entorno donde se ejecutará el job.
steps: tareas individuales.
uses: utiliza una acción existente.
run: ejecuta un comando.

Ejemplo de estructura

```
name: Integración Continua

on:
  push:
    branches: [ main ]

jobs:
  pruebas:
    runs-on: ubuntu-latest

    steps:
      - name: Descargar código
        uses: actions/checkout@v4

      - name: Ejecutar pruebas
        run: echo "Ejecutando pruebas..."
```

9. Workflow básico para Flutter

Como framework seleccionado se utilizará Flutter. El workflow se guarda en .github/workflows/flutter.yml.

```
name: Flutter CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  flutter:
```

```
    runs-on: ubuntu-latest

    steps:
      - name: Descargar código
        uses: actions/checkout@v4

      - name: Configurar Flutter
        uses: subosito/flutter-action@v2
        with:
          channel: stable

      - name: Instalar dependencias
        run: flutter pub get

      - name: Analizar código
        run: flutter analyze

      - name: Ejecutar pruebas
        run: flutter test
```

10. Acciones relacionadas con Flutter

flutter pub get: obtiene las dependencias especificadas en pubspec.yaml.
flutter analyze: analiza el código para detectar problemas.
flutter test: ejecuta las pruebas automatizadas del proyecto.

11. Realizar un cambio en el proyecto

Para comprobar el funcionamiento del pipeline, se puede modificar un texto de la aplicación, por ejemplo:

```
Text('Hola, GitHub Actions')

por:

Text('Hola, CI/CD con Flutter')
```

Después de guardar el cambio:

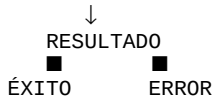
```
git add .
git commit -m "Modificar texto de la aplicación"
git push origin main
```

12. Comprobar que GitHub Actions detecte el cambio

Después de realizar el push, GitHub detecta el evento configurado en el workflow. En este caso, el evento push en la rama main activa el workflow. La ejecución puede consultarse en: Repositorio de GitHub → Actions → Flutter CI.

13. Verificación del resultado

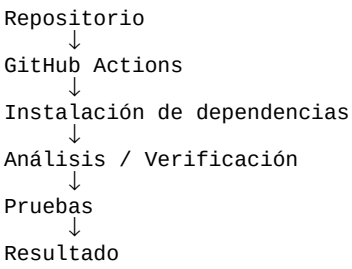
```
Cambio en el proyecto
        ↓
     git push
        ↓
Repositorio GitHub
        ↓
GitHub Actions
        ↓
Descargar código
        ↓
Configurar Flutter
        ↓
Instalar dependencias
        ↓
Analizar código
        ↓
Ejecutar pruebas
```



Si todos los pasos terminan correctamente, GitHub muestra el workflow como exitoso. Si algún paso falla, aparece como fallido y los registros (logs) permiten identificar dónde ocurrió el problema.

14. Pipeline completo

Repositorio → GitHub Actions → Instalación de dependencias → Verificación/Pruebas → Resultado



15. Conclusión

La práctica permitió comprender los conceptos fundamentales de CI/CD y su aplicación mediante GitHub Actions. Los workflows definidos en archivos YAML permiten automatizar tareas dentro de un repositorio, mientras que los jobs y steps organizan las acciones que se ejecutan. En Flutter, el pipeline puede utilizar flutter pub get, flutter analyze y flutter test para instalar dependencias, analizar el código y ejecutar pruebas. Al realizar un cambio y ejecutar git push, el workflow configurado para el evento push puede ejecutarse automáticamente, permitiendo detectar errores de manera temprana.

Bibliografía verificable

1. GitHub Docs. Descripción de Acciones de GitHub.
https://docs.github.com/es/actions/get-started/understand-github-actions?learn=getting_started

2. GitHub Docs. Sintaxis del flujo de trabajo para Acciones de GitHub.
https://docs.github.com/es/actions/reference/workflows-and-actions/workflow-syntax

3. GitHub Docs. Workflows - GitHub Actions.
https://docs.github.com/en/actions/concepts/workflows-and-actions/workflows

4. GitHub Docs. GitHub Actions documentation.
https://docs.github.com/en/actions

5. Flutter Documentation. The Flutter command-line tool.
https://docs.flutter.dev/reference/flutter-cli

Nota: Las fuentes anteriores corresponden a documentación oficial de GitHub y Flutter y pueden verificarse directamente en sus sitios web.